

Program 3: 1D Array Merging

Aim

To write a C program that merges two one-dimensional arrays into a single array.

Theory

Merging combines the elements of two arrays into a third array whose size is $m + n$ (m and n are the sizes of the two arrays).

In **simple merging**, all elements of the first array are copied, followed by all elements of the second array.

In **sorted merging**, both input arrays are already sorted. Two index variables i and j point to the current elements of the arrays; the smaller element is copied into the result and its index is advanced. Remaining elements of the longer array are then copied. Time complexity: $O(m + n)$. This is the key step of the Merge Sort algorithm.

Algorithm

1. Read the sizes m and n and the elements of arrays A and B (sorted).
2. Simple merge: copy $A[0..m-1]$ into C , then copy $B[0..n-1]$ after it.
3. Sorted merge: set $i = j = k = 0$. While $i < m$ and $j < n$, copy the smaller of $A[i]$ and $B[j]$ into $C[k]$ and advance the corresponding indices.
4. Copy any remaining elements of A , then any remaining elements of B .
5. Print the merged array.

C Source Code

```
/*
 * Program : 1D Array Merging
 *
 * THEORY:
 * - Merging combines two arrays into a single array.
 * - Simple merge : copy all elements of A, then all elements of B.
 * - Sorted merge : if A and B are already sorted, use two pointers (i, j)
 *                  and always copy the smaller current element.
 *                  Time  $O(m + n)$ . This is the core step of Merge Sort.
 * - The result array must have size  $m + n$ .
 */
#include <stdio.h>
#define MAX 100

int main() {
    int a[MAX], b[MAX], c[2 * MAX], m, n;

    printf("Enter size of first array: ");
    scanf("%d", &m);
    printf("Enter %d sorted elements: ", m);
    for (int i = 0; i < m; i++) scanf("%d", &a[i]);

    printf("Enter size of second array: ");
    scanf("%d", &n);
    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &b[i]);

    /* ---- Method 1: simple (concatenation) merge ---- */
    int k = 0;
    for (int i = 0; i < m; i++) c[k++] = a[i];
    for (int j = 0; j < n; j++) c[k++] = b[j];
}
```

```

printf("\nSimple merge : ");
for (int i = 0; i < k; i++) printf("%d ", c[i]);

/* ---- Method 2: sorted merge (two pointers) ---- */
int i = 0, j = 0;
k = 0;
while (i < m && j < n) {
    if (a[i] <= b[j]) c[k++] = a[i++];
    else c[k++] = b[j++];
}
while (i < m) c[k++] = a[i++]; /* leftovers of A */
while (j < n) c[k++] = b[j++]; /* leftovers of B */

printf("\nSorted merge : ");
for (int x = 0; x < k; x++) printf("%d ", c[x]);
printf("\n");
return 0;
}

```

Sample Input

Input: A = {1 5 9 13}, B = {2 4 6 8 20}.

Sample Output (program run with the input above)

```

Enter size of first array: Enter 4 sorted elements: Enter size of second array: Enter 5 sorted elements
Simple merge : 1 5 9 13 2 4 6 8 20
Sorted merge : 1 2 4 5 6 8 9 13 20

```